Apuntes: Clases en JavaScript

1. Qué es una clase

Una **clase** es un molde para fabricar objetos con la misma forma (propiedades) y el mismo comportamiento (métodos). Cada objeto fabricado con el molde es una **instancia**, y tiene sus propios datos.

Sin clases tendrías que escribir cada objeto a mano:

```
const alumno1 = { nombre: "Ana", nota: 8, aprobado() { return this.nota >= 5;
} };
const alumno2 = { nombre: "Luis", nota: 4, aprobado() { return this.nota >=
5; } };
// ... y así con 30 alumnos. Si cambia la nota de aprobado, hay que tocar 30
sitios.
```

Con una clase, el molde se escribe una sola vez:

```
class Alumno {
  constructor(nombre, nota) {
    this.nombre = nombre;
    this.nota = nota;
  }
  aprobado() {
    return this.nota >= 5;
  }
}

const ana = new Alumno("Ana", 8);
const luis = new Alumno("Luis", 4);

ana.aprobado();    // true
luis.aprobado();   // false
```

2. Cómo se escribe una clase

```
class Punto {
  constructor(x = 0, y = 0) {
    this.x = x;
    this.y = y;
  }
  mostrar() {
    return `(${this.x}, ${this.y})`;
  }
}

const p = new Punto(11, 15);
p.mostrar();   // "(11, 15)"
```

Pieza	Qué es	Recuerda
`class Punto`	Declara el molde	Nombre con mayúscula inicial
`constructor(...)`	Función que se ejecuta al hacer `new`	Solo puede haber uno por clase
`this`	El objeto que se está creando o usando	`this.x = x` crea la propiedad `x` en ese objeto
`mostrar() { }`	Método: una función que tienen todas las instancias	Sin la palabra `function` y sin comas entre métodos
`new Punto(11, 15)`	Crea una instancia	Sin `new` da error

Valores por defecto

En `constructor(x = 0, y = 0)`, si no se pasa un argumento, toma ese valor:

```
new Punto();       // (0, 0)
new Punto(5);      // (5, 0)
```

No es obligatorio ponerlos. Úsalos cuando haya un valor razonable (el origen, una cantidad 0). No los pongas si el dato es obligatorio: un producto sin precio no debería valer 0 sin que nadie se entere.

Si falta un argumento sin valor por defecto, vale `undefined`, y los cálculos con él dan `NaN`.

Pruébalo en la consola

Abre cualquier ejercicio, pulsa F12 y escribe:

```
const p = new Punto(3, 4);
p.x = 10;            // las propiedades se pueden cambiar
p.mostrar();         // "(10, 4)"
p instanceof Punto   // true
Punto(3, 4);         // TypeError: hace falta new
```

3. Un constructor, varias formas de llamarlo

En JavaScript una clase solo tiene **un** constructor. Para crear objetos de varias formas, el constructor mira qué le han pasado y actúa en consecuencia.

Dos reglas que hay que tener claras:

1. Los argumentos se reparten **por posición**: el 1.º va al 1.er parámetro, el 2.º al 2.º, y así.
2. Si faltan argumentos no da error: los que faltan valen `undefined` (o su valor por defecto). Si sobran, se ignoran.

Para saber qué te han pasado se usa `instanceof`, que responde a "¿este objeto se fabricó con esta clase?":

```
class Circulo {
  constructor(x = 0, y = 0, r = 0) {
    if (x instanceof Punto) {          // new Circulo(punto, r)
      this.centro = new Punto(x);
      this.r = y;                       // ¡el radio llega en el 2.º
parámetro!
    } else {                            // new Circulo(x, y, r)
      this.centro = new Punto(x, y);
      this.r = r;
    }
  }
}
```

Los nombres `x, y, r` solo describen la llamada normal. Fíjate en qué recibe cada parámetro:

Llamada	`x`	`y`	`r`	Radio guardado
`new Circulo(26, 42, 8)`	26	42	8	8 (de `r`)
`new Circulo(p, 4)`	un `Punto`	4	0 (no se usa)	4 (de `y`)

Constructor copia

Si el constructor recibe un objeto de su misma clase, puede crear una copia con los mismos datos:

```
class Punto {
  constructor(x = 0, y = 0) {
    if (x instanceof Punto) {   // new Punto(otroPunto)
      this.x = x.x;
      this.y = x.y;
    } else {
      this.x = x;
      this.y = y;
    }
  }
}

const p1 = new Punto(11, 15);
const p2 = new Punto(p1);   // copia
p1.x = 69;
p2.x;                       // 11 → p2 no cambia
```

4. Composición: un objeto que *tiene* otro

Componer es guardar un objeto dentro de otro como propiedad. El objeto de fuera **delega** en el de dentro lo que este ya sabe hacer.

Ejemplo: un rectángulo tiene dos puntos

Un rectángulo con los lados paralelos a los ejes queda definido por **dos esquinas opuestas**. Es lo mismo que haces al dibujar un rectángulo en Paint: pulsas en una esquina y arrastras hasta la contraria. Las otras dos esquinas salen de cruzar las coordenadas.

```
 y
11 ·  (5,11) ●──────────● (9,11)  ← p2
   |         |          |
 7 ·  (5,7)  ●──────────● (9,7)
   |       ↑ p1
   └──────────────────────── x
           5           9
```

- Base = distancia en horizontal = |9 − 5| = 4
- Altura = distancia en vertical = |11 − 7| = 4
- `Math.abs` hace que no importe el orden en que te den las esquinas.

```
class Rectangulo {
  constructor(p1, p2) {
    this.p1 = p1;   // el rectángulo TIENE dos puntos
    this.p2 = p2;
  }
  base()   { return Math.abs(this.p2.x - this.p1.x); }
  altura() { return Math.abs(this.p2.y - this.p1.y); }
  area()   { return this.base() * this.altura(); }
}

const r = new Rectangulo(new Punto(5, 7), new Punto(9, 11));
r.area();   // 16
```

Ejemplo: una caja tiene un rectángulo

```
class Caja {
  constructor(rectangulo, profundidad = 0) {
    this.rectangulo = rectangulo;   // la caja TIENE un rectángulo
    this.profundidad = profundidad;
  }
  volumen() {
    return this.rectangulo.area() * this.profundidad;   // delega en el
rectángulo
  }
}

const caja = new Caja(r, 10);
caja.volumen();   // 160
```

La caja no sabe calcular el área de un rectángulo, ni le hace falta: se lo pide a su rectángulo.

Ejemplo sin geometría: un carrito tiene un cliente

```
class Carrito {
  constructor(cliente) {
    this.cliente = cliente;   // TIENE un cliente
    this.productos = [];      // TIENE varios productos
  }
  agregar(producto) {
    this.productos.push(producto);
  }
}
```

5. Referencias y copias

Al asignar un objeto a otra variable **no se copia**: las dos variables apuntan al mismo objeto.

```
const a = { x: 1 };
const b = a;     // b es otro nombre para el MISMO objeto
b.x = 5;
a.x;             // 5  ← a también ha cambiado
```

Para tener un objeto independiente hay que crear uno nuevo con `new`:

```
const p1 = new Punto(11, 15);

const p2 = p1;              // REFERENCIA: el mismo objeto
const p3 = new Punto(p1);   // COPIA: un objeto nuevo con los mismos datos

p1.x = 69;
p2.x;   // 69
p3.x;   // 11
```

En la composición

Cuando un objeto guarda otro, decides si guarda una referencia o una copia:

```
this.centro = x;              // referencia: si cambia el punto original,
cambia el círculo
this.centro = new Punto(x);   // copia: el círculo tiene su propio centro
```

Ninguna es incorrecta. Lo importante es saber cuál estás haciendo. Si no quieres que un cambio desde fuera afecte a tu objeto, guarda una copia.

6. Herencia: un objeto que *es* otro

Con `extends`, una clase hija recibe todas las propiedades y métodos de la clase padre, y añade o cambia lo que necesite.

```
class Persona {
  constructor(nombre, edad) {
    this.nombre = nombre;
    this.edad = edad;
  }
  presentarse() {
    return `Soy ${this.nombre} y tengo ${this.edad} años`;
  }
}
```

```
class Alumno extends Persona {        // un alumno ES una persona
  constructor(nombre, edad, curso) {
    super(nombre, edad);              // 1.º: el padre rellena su parte
    this.curso = curso;               // 2.º: lo que añade el hijo
  }
  presentarse() {                     // REDEFINIDO
    return `${super.presentarse()} y estudio ${this.curso}`;
  }
}

const ana = new Alumno("Ana", 19, "2.º DAW");
ana.nombre;          // "Ana"  ← heredado de Persona
ana.presentarse();   // "Soy Ana y tengo 19 años y estudio 2.º DAW"
ana instanceof Persona;   // true
```

Las cuatro ideas clave

1. **`extends`**: indica de qué clase se hereda.
2. **`super(...)`** en el constructor: llama al constructor del padre. Es obligatorio y va **antes** de usar `this`; si no, da `ReferenceError`.
3. **Redefinir un método**: escribir en el hijo un método con el mismo nombre que en el padre. Se usa el del hijo.
4. **`super.metodo()`**: desde el hijo, llama a la versión del padre para reutilizarla.

Ejemplo: redefinir para cambiar un comportamiento

```
class Producto {
  constructor(nombre, precio) {
    this.nombre = nombre;
    this.precio = precio;
  }
  iva() { return 0.21; }
  precioFinal() { return this.precio * (1 + this.iva()); }
}

class Libro extends Producto {
  iva() { return 0.04; }   // solo cambia el IVA
}

new Producto("Auriculares", 50).precioFinal();   // 60.5
new Libro("El Quijote", 20).precioFinal();       // 20.8
```

`Libro` no reescribe `precioFinal()`, pero calcula bien su precio: `precioFinal()` llama a `this.iva()`, y para un libro `this.iva()` es la versión de `Libro`. A esto se le llama **polimorfismo**: el mismo mensaje da respuestas distintas según el objeto.

Como `Libro` no tiene constructor propio, usa el de `Producto`. Solo hace falta escribir uno si el hijo añade datos.

Si el constructor admite varias formas

Cuando uses `instanceof` en un constructor con herencia, pregunta primero por la clase **más concreta**. Un `Circulo` que hereda de `Punto` también es `instanceof Punto`, así que si preguntas antes por `Punto`, nunca llegarás a la rama de `Circulo`.

7. ¿Composición o herencia?

Di la frase en voz alta y comprueba si es verdad **siempre**:

Frase	¿Verdad?	Se usa
Un alumno **es una** persona	Sí	Herencia: `Alumno extends Persona`
Un libro **es un** producto	Sí	Herencia: `Libro extends Producto`
Un carrito **es un** cliente	No: **tiene** un cliente	Composición: `this.cliente`
Un rectángulo **es un** punto	No: **tiene** dos puntos	Composición: `this.p1`, `this.p2`
Una caja **es un** rectángulo	No: **tiene** uno como base	Composición: `this.rectangulo`

Segunda prueba: ¿puedo usar el hijo en cualquier sitio donde se espera el padre? Un libro se puede meter en un carrito como cualquier producto. Un carrito no se puede usar donde se espera un cliente: ¿le enviarías una factura a un carrito?

	Composición	Herencia
Relación	*tiene un*	*es un*
Código	`this.cliente = cliente`	`class Libro extends Producto`

	Composición	Herencia
Acceso a los datos	`carrito.cliente.nombre`	`libro.nombre`
Reutiliza código	Delegando: `this.rectangulo.area()`	Heredando: `super.metodo()`

La herencia no se elige porque funcione o porque ahorre líneas, sino porque la frase "X es un Y" sea verdad. Ante la duda, usa composición.

8. Si vienes de Java

Las ideas son las mismas; cambia cómo se escriben.

	Java	JavaScript
Constructores	Varios (sobrecarga)	Uno solo; se decide dentro con `instanceof`
Argumentos que faltan o sobran	Error de compilación	Valen `undefined` o se ignoran
Atributos	Se declaran con tipo (`private int r;`)	No se declaran: se crean con `this.r = ...`
Atributos privados	`private`	`#r` (o nada)
`extends` y `super()`	Igual	Igual; `super()` antes de `this`
Redefinir un método	`@Override`	Basta con usar el mismo nombre
Interfaces y clases abstractas	Sí	No existen

9. Errores frecuentes

Abre siempre la consola (F12): el error dice qué falla y en qué línea.

Lo que ves	Por qué pasa	Cómo arreglarlo
`undefined` en pantalla	El método no tiene `return`	Devuelve el resultado con `return`

Lo que ves	Por qué pasa	Cómo arreglarlo
`NaN`	Usas `r` en vez de `this.r`, o falta un argumento	Las propiedades siempre con `this.`
`X is not defined`	La clase no existe o está mal escrita	Revisa mayúsculas y nombre
`... is not a function`	El método no existe o se llama distinto (`Area` y `area`)	Compara con la llamada
Área negativa	Restas coordenadas sin `Math.abs`	`Math.abs(p2.x - p1.x)`
`Must call super constructor`	Usas `this` antes de `super()`	`super(...)` en la primera línea del constructor
Un valor sale 0 al copiar	Preguntas por la clase padre antes que por la hija	`instanceof` de la más concreta a la más general
`SyntaxError` en la clase	Comas entre métodos o `function` delante	Los métodos van sin comas y sin `function`
Se cambia un objeto sin tocarlo	Guardaste una referencia, no una copia	Copia con `new Clase(objeto)`

10. Chuleta

```
// Clase con constructor, valores por defecto y método
class Persona {
  constructor(nombre, edad = 0) {
    this.nombre = nombre;
    this.edad = edad;
  }
  presentarse() {
    return `Soy ${this.nombre}`;
  }
}

// Instancia
const p = new Persona("Ana", 19);

// Composición: TIENE un objeto
class Grupo {
  constructor(tutor) {
    this.tutor = tutor;          // referencia
```

```
    // this.tutor = new Persona(tutor.nombre, tutor.edad);   // copia
  }
}

// Herencia: ES un objeto
class Alumno extends Persona {
  constructor(nombre, edad, curso) {
    super(nombre, edad);         // siempre lo primero
    this.curso = curso;
  }
  presentarse() {                // redefinido
    return super.presentarse() + `, de ${this.curso}`;
  }
}

// Comprobar el tipo
new Alumno("Ana", 19, "2.º DAW") instanceof Persona;   // true
```

Quiero...	Escribo
Crear un objeto	`new Clase(...)`
Guardar un dato en el objeto	`this.dato = valor`
Saber de qué clase es	`objeto instanceof Clase`
Que un objeto contenga otro	`this.otro = objeto` (composición)
Heredar de otra clase	`class Hija extends Padre`
Inicializar la parte del padre	`super(...)` en la 1.ª línea del constructor
Usar un método del padre desde el hijo	`super.metodo()`
Copiar un objeto	`new Clase(objetoOriginal)` (si la clase tiene constructor copia)